# Lösungen der Warm-up-Karten

Gleiche Nummerierung wie auf den Karten. Fachlich richtige Alternativen sind gleichwertig – entscheidend sind die genannten Kernpunkte.

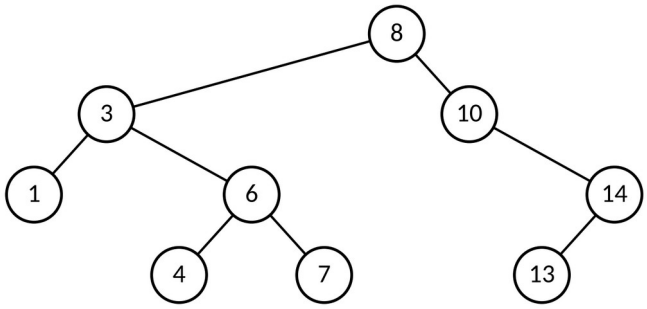
## W5-01 Der Setzling · Objektkarten und Referenzen

a) siehe Baum – es entsteht genau der beispielBST der Baumschule. Kontrolle: Preorder 8 3 1 6 4 7 10 14 13.

b) Wurzel 8; Blätter 1, 4, 7, 13; innere Knoten 8, 3, 6, 10, 14; Höhe 4 (vier Ebenen); Knoten 7 liegt auf Ebene 3 (Weg 8 → 3 → 6 → 7).

c) Zeile 6: b.getRight() ist das Blatt 10, dessen rechter Teilbaum ein leerer Baum (nicht null!) ist. setItem auf einem leeren Baum macht ihn zum Blatt mit zwei leeren Teilbäumen – ein Knoten 14 entsteht. Zeile 7: t ist eine zweite Referenz auf denselben Teilbaum (Knoten 6), keine Kopie. setLeft/setRight über t ändern deshalb den Baum, den auch b erreicht.

d) b.getLeft().getLeft() ist das Blatt 1; dessen getLeft() ist ein leerer Baum. getItem() auf dem leeren Baum → Laufzeitfehler (IllegalStateException). Absichern mit if (!x.isEmpty()) bzw. vorher isLeaf() prüfen.




*Baum b nach Zeile 10*

## W5-02 Wegweiser im Wald · Methodenkopf und Signatur

a) siehe Code. BinTree bleibt BinTree (kein Inhaltstyp in Java), „dynamische Reihung“ heißt in Java DynArray.

b) spiegeln(b: BinTree): nichts · baueAus(r: Reihung von Ganzzahl): BinTree (genauer: BinTree vom Inhaltstyp Ganzzahl).

c) Zeile 1: Fehler – getItem() liefert Object, Object → int geht nicht ohne Cast; richtig: int w = (Integer) b.getItem(); Zeile 2: in Ordnung – der Methodenaufruf bindet stärker als der Cast, gecastet wird also das Ergebnis von getItem(), danach wird 1 addiert. Zeile 3: akzeptiert, scheitert aber zur Laufzeit, wenn der linke Teilbaum leer ist (IllegalStateException) – vorher !b.getLeft().isEmpty() prüfen. Zeile 4: in Ordnung. Zeile 5: in Ordnung (Zeichenketten immer mit equals vergleichen). Zeile 6: Fehler – getLeft() liefert einen BinTree, kein Inhaltselement; ein BinTree lässt sich nicht in Integer umwandeln. Richtig: int z = (Integer) b.getLeft().getItem(); (mit Leer-Abfrage).

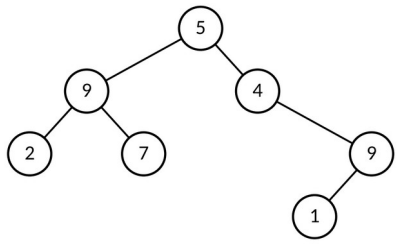
```
public static int maximum(BinTree b)
public static String pfadZu(BinTree b, int x)
public static DynArray blaetter(BinTree b)
public static boolean istGleich(BinTree a, BinTree b)
```

## W5-03 Drei Wege durch den Wald · Funktionsweise erläutern

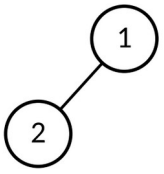
a) Preorder 8 3 1 6 4 7 10 14 13 · Inorder 1 3 4 6 7 8 10 13 14 (beim BST aufsteigend!) · Postorder 1 4 7 6 3 13 14 10 8.

b) Wurzel 5; Inorder links davon 2 9 7, rechts 4 1 9. Links: Preorder 9 2 7 → Wurzel 9 mit 2 links und 7 rechts. Rechts: Preorder 4 9 1 → Wurzel 4; in der Inorder 4 1 9 steht nichts links von 4, also ist der linke Teilbaum leer; rechts bleibt 1 9 mit Preorder 9 1 → Wurzel 9, 1 links. B ist der zahlenbaum der Baumschule. Postorder 2 7 9 1 9 4 5.

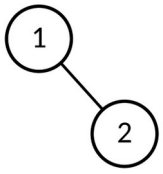
c) Wurzel 1 mit 2 als linkem Kind bzw. 2 als rechtem Kind: beide Bäume haben Preorder 1 2 und Postorder 2 1. Erst die Inorder (2 1 bzw. 1 2) unterscheidet sie.




*Baum B (rekonstruiert)*




*Baum c) links*



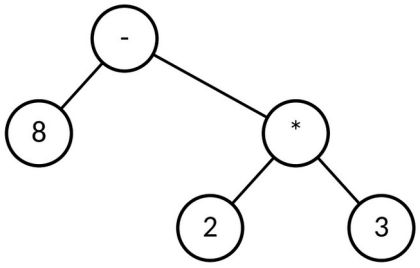

*Baum c) rechts*

## W5-04 Der Rechenbaum · Lückencode

a) Preorder * + 3 4 - 5 2 · Inorder 3 + 4 * 5 - 2 · Postorder 3 4 + 5 2 - * (= Postfix-Notation; der Stapel-Rechner aus Welt 4 berechnet daraus 21). Die Inorder 3 + 4 * 5 − 2 verliert die Struktur: Nach Punkt vor Strich würde sie als 3 + (4 * 5) − 2 gelesen (hier zufällig auch 21; beim Baum zu (3 + 4) * 5 käme aber 23 statt 35 heraus). Deshalb setzt man bei der Inorder um jeden inneren Knoten Klammern: ((3 + 4) * (5 − 2)).

b) Die Wurzel ist der Operator, der zuletzt ausgeführt wird: "-" mit 8 links und dem Teilbaum "*" (2, 3) rechts. Wert 8 − 6 = 2.

c) (1) (String) – getItem() liefert Object. (2) t.isLeaf() – Blätter sind Zahlen (t ist nie leer, weil jeder Operator zwei Operanden hat). (3) t.getLeft() (4) s.equals("-") – Zeichenketten mit equals vergleichen, nicht mit ==. (5) l / r – ganzzahlige Division. berechne(T) liefert 21.




*Baum 8 − 2 * 3*

```
(1) (String)   (2) t.isLeaf()   (3) t.getLeft()
(4) s.equals("-")   (5) l / r
```

## W5-05 Wie hoch ist der Baum? · Tracetabelle

a) siehe Tabelle: 7 Aufrufe, Ergebnis 3. Der linke Teilbaum von 4 ist leer – auch dafür gibt es einen Aufruf (Rückgabe 0). Ein Aufruf ist erst beendet, wenn beide Teilaufrufe zurückgekehrt sind (Rückgabewert von unten nach oben lesen).

b) Jeder der n Knoten erzeugt einen Aufruf; jeder Knoten hat zwei Teilbäume, also gibt es insgesamt 2n Teilbäume, von denen n leer sind, + 1 für die Wurzel → n + (n + 1) = 2n + 1 Aufrufe. C: 2 · 3 + 1 = 7 ✓. zahlenbaum: 2 · 7 + 1 = 15.

c) siehe Code: Basisfall leer → 0; Blatt → 1; sonst Summe der Blätter beider Teilbäume (für C: 1, für den zahlenbaum: 3).

```
Rekursionstiefe | Aufruf                    | Rückgabewert
----------------+---------------------------+-------------
       0        | hoehe(4)                  |      3
       1        | hoehe(leer)  links von 4  |      0
       1        | hoehe(9)                  |      2
       2        | hoehe(1)                  |      1
       3        | hoehe(leer)  links von 1  |      0
       3        | hoehe(leer)  rechts von 1 |      0
       2        | hoehe(leer)  rechts von 9 |      0
```

```
public static int anzahlBlaetter(BinTree b) {
    if (b.isEmpty()) {
        return 0;
    }
    if (b.isLeaf()) {
        return 1;
    }
    return anzahlBlaetter(b.getLeft()) + anzahlBlaetter(b.getRight());
}
```

## W5-06 Der Ast bricht · Fehler im Java-Code

a) Zeilen 3 und 5: getItem() liefert Object; ein Object kann nicht zurückgegeben bzw. addiert werden, wo int erwartet wird. Korrektur: return (Integer) b.getItem(); bzw. return (Integer) b.getItem() + summe(…) + summe(…);

b) summe(5) → summe(9) → summe(2) (Blatt: 2) → summe(7) (Blatt: 7) → summe(4) → summe(linker Teilbaum von 4) – dieser ist leer: isLeaf() auf einem leeren Baum → IllegalStateException. Ursache: Der Basisfall „Blatt“ deckt nicht alle Fälle ab – ein Knoten mit genau einem Kind (hier 4) führt zu einem Aufruf mit dem leeren Baum. Auch summe(leerer Baum) scheitert sofort.

c) Basisfall immer „leerer Baum“: if (b.isEmpty()) return 0; – dann braucht man keinen Sonderfall für Blätter. summe(zahlenbaum) = 5 + 9 + 2 + 7 + 4 + 9 + 1 = 37.

```
public static int summe(BinTree b) {
    if (b.isEmpty()) {
        return 0;
    }
    return (Integer) b.getItem() + summe(b.getLeft()) + summe(b.getRight());
}
```

## W5-07 Zählen auf Ebene k · Code ↔ Struktogramm

a) Ebene 0 → 1 (5), Ebene 1 → 2 (9, 4), Ebene 2 → 3 (2, 7, 9), Ebene 3 → 1 (1), Ebene 4 → 0. Bei jedem Abstieg wird ebene um 1 kleiner; gezählt wird ein Knoten, wenn ebene dort 0 ist.

b) siehe Struktogramm. Die Reihenfolge „erst leer?, dann ebene = 0?“ muss erhalten bleiben.

c) Ebene 2 → 4, Ebene 4 → 2. Jetzt wird auch ein leerer Teilbaum auf der gesuchten Ebene mit 1 gezählt (auf Ebene 2 der leere linke Teilbaum von 4, auf Ebene 4 die beiden leeren Teilbäume des Blatts 1). Einen Laufzeitfehler gibt es nicht (isEmpty wird vor getLeft weiterhin geprüft) – der Fehler fällt nur durch falsche Werte auf, deshalb gehört der Basisfall „leer“ immer an den Anfang.

```
anzahlAufEbene(b: BinTree, ebene: Ganzzahl): Ganzzahl
  falls b.isEmpty()
    dann: gib 0 zurück
    sonst: –
  falls ebene = 0
    dann: gib 1 zurück
    sonst: –
  l ← anzahlAufEbene(b.getLeft(), ebene – 1)
  r ← anzahlAufEbene(b.getRight(), ebene – 1)
  gib l + r zurück
```

## W5-08 Blätter sammeln · Methode implementieren

a) siehe Code (wie im L5-Gerüst; sammleInorder(b, d) ist genauso aufgebaut). Basisfall: leerer Baum → nichts tun. Ein Blatt wird angehängt; sonst erst links, dann rechts sammeln (so entsteht die Reihenfolge von links nach rechts).

b) zahlenbaum → 2, 7, 1 · beispielBST → 1, 4, 7, 13.

c) Alle Aufrufe sollen in dieselbe Reihung schreiben (dasselbe Objekt, die Referenz wird übergeben). Würde jeder Aufruf eine neue DynArray erzeugen, gingen die gesammelten Werte verloren – oder man müsste die Ergebnisse der Teilaufrufe mühsam zusammenkopieren.

```
public static DynArray blaetter(BinTree b) {
    DynArray d = new DynArray();
    blaetter(b, d);
    return d;
}
```

```
public static void blaetter(BinTree b, DynArray d) {
    if (!b.isEmpty()) {
        if (b.isLeaf()) {
            d.append(b.getItem());
        } else {
            blaetter(b.getLeft(), d);
            blaetter(b.getRight(), d);
        }
    }
}
```

## W5-09 Die Wanderschlange · Tracetabelle

a) siehe Tabelle. Ergebnis "8 3 10 1 6 14 4 7 13 " – erst Ebene 0, dann Ebene 1, … (Schlange: zuerst hineingelegt, zuerst entnommen).

b) dequeue() liefert Object. Um getItem(), getLeft() und getRight() aufrufen zu können, muss t den Typ BinTree haben.

c) Stapel: "8 10 14 13 3 6 7 4 1 " – der zuletzt abgelegte (rechte) Teilbaum wird zuerst bearbeitet. Mit vertauschten if-Anweisungen (erst rechts, dann links ablegen) entsteht die Preorder "8 3 1 6 4 7 10 14 13 " – ein Stapel ersetzt die Rekursion.

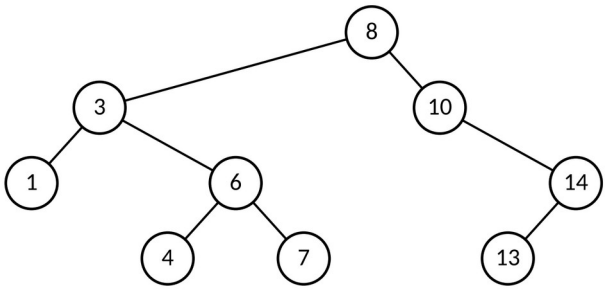
```
entnommen | Schlange danach (vorn zuerst) | s
----------+-------------------------------+--------------------
    8     | 3, 10                         | 8
    3     | 10, 1, 6                      | 8 3
    10    | 1, 6, 14                      | 8 3 10
    1     | 6, 14                         | 8 3 10 1
    6     | 14, 4, 7                      | 8 3 10 1 6
    14    | 4, 7, 13                      | 8 3 10 1 6 14
    4     | 7, 13                         | 8 3 10 1 6 14 4
    7     | 13                            | 8 3 10 1 6 14 4 7
    13    | –                             | 8 3 10 1 6 14 4 7 13
```

## W5-10 Die Item-Datenbank wächst · Funktionsweise erläutern

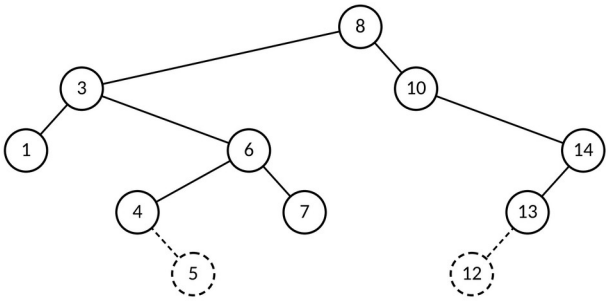
a) siehe Baum (beispielBST): 3 < 8 → links, 10 > 8 → rechts, 1 → 8, 3 → links von 3, 6 → rechts von 3, 14 → rechts von 10, 4 → 8, 3, 6 → links von 6, 7 → rechts von 6, 13 → 8, 10, 14 → links von 14.

b) 5: 8 (kleiner) → 3 (größer) → 6 (kleiner) → 4 (größer) → rechter Teilbaum von 4 ist leer → 5 wird rechtes Kind von 4. 12: 8 → 10 → 14 → 13 → 12 wird linkes Kind von 13. Jeweils 4 Vergleiche.

c) 1 3 4 5 6 7 8 10 12 13 14. Inorder besucht zuerst den ganzen linken Teilbaum (alle kleiner), dann die Wurzel, dann den rechten Teilbaum (alle größer) – und das gilt rekursiv für jeden Teilbaum. Einfügen von 6: Suchweg 8 → 3 → 6, der Wert ist schon vorhanden → nichts passiert.




*Baum nach a)*




*Baum nach b) – neu: 5 und 12*

## W5-11 Suchen mit Zählwerk · Struktogramm lesen

a) x = 7: 8, 3, 6, 7 → 4 · x = 5: 8, 3, 6, 4 → danach leerer Teilbaum → 4 (nicht gefunden) · x = 13: 8, 10, 14, 13 → 4 · x = 8: 8 → 1.

b) 4 = Höhe des Baums: Man steigt je Vergleich eine Ebene tiefer, mehr Ebenen gibt es nicht. Der Rückgabewert ist also höchstens die Höhe (längster Weg von der Wurzel zu einem Blatt in Knoten).

c) Im BST liegen alle kleineren Werte links und alle größeren rechts. Ist x < w, kann x nur im linken Teilbaum liegen – der rechte wird gar nicht betrachtet. Ohne Suchbaum-Ordnung kann x überall stehen: Im schlimmsten Fall (x nicht vorhanden) müssen alle Knoten untersucht werden, beim zahlenbaum alle 7.

## W5-12 Ein Item verschwindet · Verändern und Erweitern

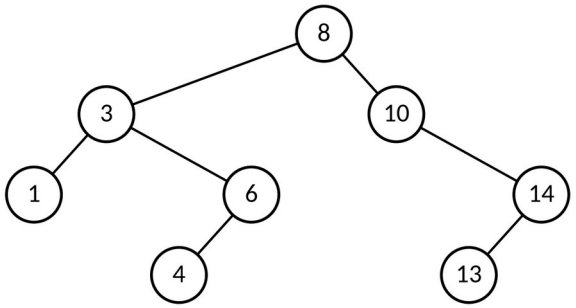
a) Fall 1: 7 ist ein Blatt → 6 hat danach nur noch das linke Kind 4.

b) Fall 2: 10 hat nur das rechte Kind 14 → der Teilbaum 14 (mit 13) wird rechtes Kind von 8.

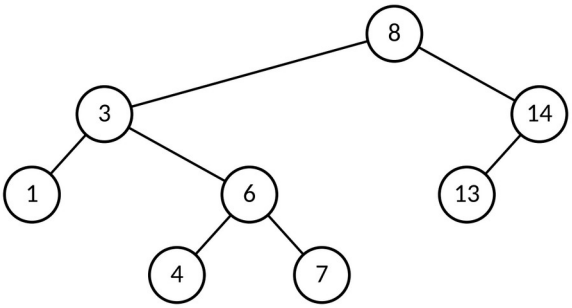
c) Fall 3: Nachfolger von 3 ist 4 (rechts zu 6, dann links zu 4). 4 ersetzt 3, das Blatt 4 wird entfernt.

d) Fall 3: Nachfolger von 8 ist 10 (rechts zu 10, links ist leer). 10 wird neue Wurzel; der alte Knoten 10 wird nach Fall 2 gelöscht, 14 rückt hoch.

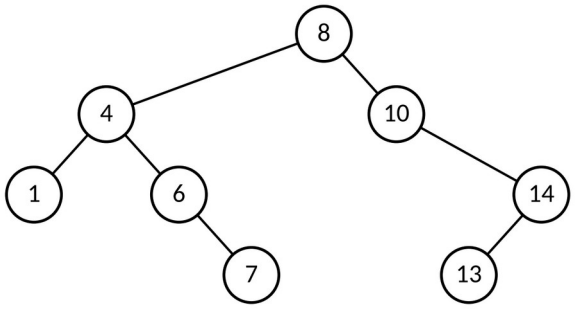
e) Vorgänger: in c) 1, in d) 7. Der Nachfolger ist der am weitesten links stehende Knoten im rechten Teilbaum – er hat kein linkes Kind (sonst wäre dieses kleiner), also höchstens ein (rechtes) Kind. Deshalb lässt er sich mit Fall 1 oder 2 entfernen. Kontrolle: Die Inorder bleibt jeweils aufsteigend sortiert.



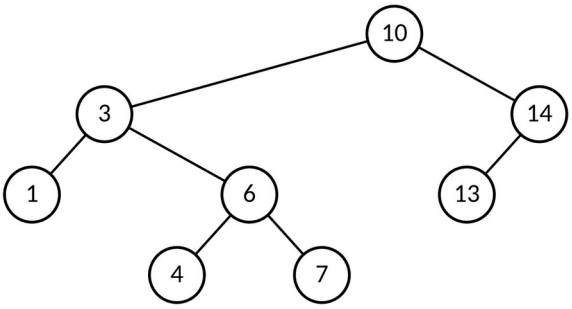

*Baum a) ohne 7*




*Baum b) ohne 10*




*Baum c) ohne 3*



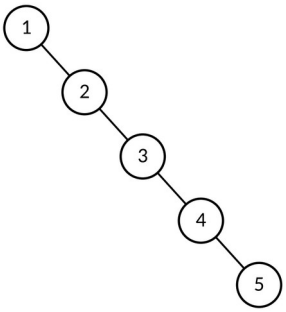

*Baum d) ohne 8*

## W5-13 Krumm oder buschig? · Funktionsweise erläutern

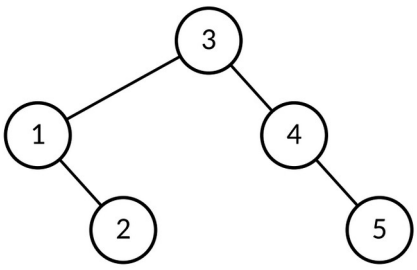
a) Einfügen von 1 bis 5: jeder neue Wert ist größer als alle bisherigen und landet ganz rechts → Kette, Höhe 5 (entartet, wie eine Liste). ausgeglichenAus: mitte = 2 → Wurzel 3; links {1, 2}: mitte 0 → 1 mit rechtem Kind 2; rechts {4, 5}: mitte 3 → 4 mit rechtem Kind 5. Höhe 3.

b) entartet: 7, 15, 31, 1023 (= n). ausgeglichen: 3, 4, 5, 10 (vollständige Bäume mit 2^h − 1 Knoten). Verdopplung von n: beim entarteten Baum doppelt so viele Vergleiche, beim ausgeglichenen nur ein Vergleich mehr (eine Ebene mehr bietet Platz für doppelt so viele Knoten) – logarithmisches Wachstum wie bei der binären Suche.

c) Jede neue Nummer ist größer als alle vorhandenen und wird ganz rechts angehängt: Der Baum wird zur rechten Kette, die Suche braucht so viele Vergleiche wie eine lineare Suche. Abhilfe: Items sammeln, in Inorder (sortiert) auslesen und mit ausgeglichenAus neu aufbauen; oder in zufälliger Reihenfolge einfügen (bzw. selbst ausgleichende Bäume – nicht Abi-Stoff).




*Baum entartet (Höhe 5)*




*Baum ausgeglichenAus (Höhe 3)*

## W5-14 Die isBST-Falle · Fehler im Java-Code

a) istBSTfalsch(D) liefert true: Jedes Kind steht für sich auf der richtigen Seite seines Elternknotens (9 > 6). Aber 9 liegt im linken Teilbaum der Wurzel 8 und ist größer als 8 → D ist kein BST (Inorder 1 3 4 6 9 8 10 13 14 ist nicht sortiert; eine Suche nach 9 ginge rechts von 8 und fände sie nicht).

b) Die Methode vergleicht nur Elternknoten und direkte Kinder. Die BST-Bedingung gilt aber für ganze Teilbäume: Jeder Knoten muss zu allen Vorfahren passen. Auf dem Weg nach unten werden die erlaubten Grenzen immer enger.

c) siehe Code. Grenzen (min, max): 8 in (−1, keine) ✓ → 3 in (−1, 8) ✓ → 6 in (3, 8) ✓ → 9 in (6, 8) ✗ → false. Beim Abstieg nach links wird der Knotenwert die neue Obergrenze, nach rechts die neue Untergrenze. Für den beispielBST liefert istBST true.

```
public static boolean istBST(BinTree b) {
    return istBST(b, -1, -1);
}
```

```
// min = -1: keine Untergrenze, max = -1: keine Obergrenze
public static boolean istBST(BinTree b, int min, int max) {
    if (b.isEmpty()) {
        return true;
    }
    int w = (Integer) b.getItem();
    if (w <= min) {
        return false;
    }
    if (max != -1 && w >= max) {
        return false;
    }
    return istBST(b.getLeft(), min, w) && istBST(b.getRight(), w, max);
}
```